

AI 데이터 분석 · 14일차

견고한 예외처리와 파이프라인

배운 부품을 조립해 한 대의 기계를 만든다

finally · continue · bare except · as e · raise · 읽기 → 분류 → 통계 → 방어 → 저장

교안 09_02(견고한 예외처리) · 09_03(데이터 처리 파이프라인)

지금까지 배운 모든 문법이 한 프로그램에 모이는 날

2026-08-07 AI 데이터 분석 교육 정리 (14일차)

학습자 Minho · 국립금오공과대학교 컴퓨터공학부

이 책을 읽는 법

어제 try-except로 오류를 받아내는 법을 배웠습니다. 오늘은 그것을 견고하게 만드는 법(09_02)과, 지금까지 배운 전부를 모아 하나의 분석 프로그램을 조립하는 법(09_03)을 배웁니다. 교안 표현으로 "새 문법은 없다 — 이미 다 배운 부품을 조립해 작동하는 한 대의 기계를 만드는 것"입니다.

키워드 finally · continue · raise **원칙** bare except 회피 · 4원칙 **문법** as e **파이프라인** 읽기·분류·통계·방어·저장 **CSV** ICT
검사 실패데이터

연결

09_03은 **종합 실습**입니다. 변수·조건문·반복문·함수·파일 입출력·예외처리가 전부 한 프로그램에 들어갑니다. 교안이 이렇게 표현했습니다 — "완성된 분석 프로그램은 그 자체로 작품이 되어 취업 준비 자산으로 남는다." 실제로 6단계를 다 만들면 포트폴리오에 올릴 만한 코드가 됩니다.

주의

이 회차는 **오전 녹취만** 남아 있습니다(STT1만 있고 407줄이 점심 안내로 끝납니다). 오전에 확인된 것은 finally · 반복문 continue · 파이프라인 도입이고, bare except · as e · raise는 **교안과 제출 실습 코드**에 근거했습니다. 배포 파일 3·4번도 확보하지 못했습니다.

목차

	장 / STEP	무엇을 배우나
CHAPTER 0	오늘의 지도 — 견고하게, 그리고 조립한다	ICT 검사 실패데이터
STEP 1	else 와 finally	네 블록의 실행 경로
STEP 2	리소스 정리	finally 와 with open
STEP 3	반복문 안에서의 예외처리	continue — 가장 실용적인 패턴
STEP 4	bare except 의 위험	좋은 예외처리 4원칙
STEP 5	as e — 에러 정보 꺼내 쓰기	무엇이 왜 실패했는지 기록
STEP 6	raise — 규칙을 강제하기	파이썬이 못 잡는 우리만의 기준
STEP 7	데이터 처리 파이프라인 설계	다섯 단계 · 나누는 이유
STEP 8	파이프라인 1~3단계	읽기 · 분류 · 통계
STEP 9	파이프라인 4~6단계	방어 · 리포트 · 검증
STEP 10	내 코드 점검 — 수업 코드 4개	도달 불가 except · 이름 불일치
STEP 11	실습 점검 — 09_02 · 09_03	파이프라인 완주 확인
부록 A·B·C	치트시트 · 오류 사전 · 예고	한 장 요약 · 함정 모음

CHAPTER 0

오늘의 지도

견고하게, 그리고 조립한다

오늘은 두 교안이 한 흐름으로 이어집니다. **09_02**가 예외처리를 실무 수준으로 다듬고, **09_03**이 그 예외처리를 포함한 모든 문법을 하나의 프로그램으로 조립합니다.

교안	다루는 것	이 책의 STEP
09_02 견고한 예외처리	else · finally · 리소스 정리 · 반복문 예외처리 · bare except · as e · raise	STEP 1~6
09_03 데이터 처리 파이프라인	파이프라인 설계 + 6단계 종합 실습	STEP 7~9

이 단원의 실데이터 — ICT 검사

두 교안이 같은 데이터를 씁니다. **ICT 검사**는 제조 부품의 전기 특성을 측정해 규격 안에 드는지 판정하는 공정입니다.

항목	내용
공정	ICT — 부품의 전기 특성을 측정해 규격 판정
기록	부품명별로 측정값 · 기준값 · 상한치 · 하한치 · 검사결과
현실	실제 현장 데이터에는 빈칸(결측) · 형식 오류 가 섞여 있다
그래서	형변환 · 인덱스 오류가 난다 → try · except로 불량 줄만 건너뛰다
파일	Data/09_ict_inspection_dirty.csv

dirty CSV — 일부러 더럽혀 둔 데이터

검사ID, 부품명, 측정값, 기준값, 상한치, 하한치, 검사결과

2197,E2,0.0,5000.0,2310.0,-3750.0,OK

9001,E2,결측,5000.0,2310.0,-3750.0,OK

← 문자가 섞임

9002,F2,,300.0,600.0,-450.0,OK

← 빈칸

9003,B2,-999,150.0,300.0,-225.0,OK

← 센서 오류값

9004,A2,128.5

← 열이 모자람

-2,,,,,,OK

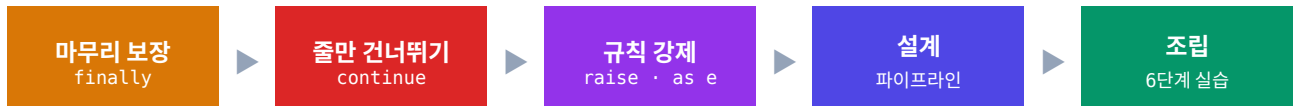
← 거의 빈 행

설비 적용

이 파일은 일부러 더럽혀 놓은 것입니다. 문자·빈칸·센서 오류값(-999)·열 부족·빈 행 — 현장에서 실제로 만나는 다섯 가지 불량 유형이 골고루 들어 있습니다. 어제 배운 `try-except` 하나로는 부족하고, 오늘 배울 `continue`·`raise`·`as e`가 각각 다른 유형을 말합니다. 데이터가 지저분한 것이 문제가 아니라, 지저분한 데이터를 건디는 코드를 못 짜는 것이 문제라는 관점이 오늘의 출발점입니다.

오늘의 흐름

앞의 세 칸이 09_02(견고하게), 뒤의 두 칸이 09_03(조립하기)입니다.



STEP 1

else 와 finally

네 블록의 실행 경로

어제 배운 try/except에 두 형제가 더 붙습니다. else는 성공했을 때만, finally는 성공하든 실패하든 무조건 **마지막에** 실행됩니다.

키워드 finally 무조건 마지막에 실행

정의 try가 성공하든 실패하든 **반드시 실행**되는 블록입니다. "마지막으로"라는 뜻 그대로입니다.

형태 try: ... except ...: ... finally: **마무리 작업**

```
text = "62.5"          # 정상

try:
    temp = float(text)
finally:
    print("종료")       # 성공·실패 무조건 실행

text = "abc"           # 비정상 → 예외가 나도
# → 역시 "종료"가 출력된 뒤 프로그램이 멈춘다

▶ 종료
```

왜 쓰나 이 성질 덕분에 **파일 닫기처럼 반드시 처리해야 할 마무리 작업**에 자주 씁니다. 예외가 나서 중간에 튕겨 나가도 finally만은 지나가기 때문입니다.

응용 except와 함께 쓰면 세 갈래가 모두 커버됩니다. 오늘 수업 코드가 정확히 그 형태입니다 — 성공하면 그대로, 실패하면 안내를 띄우고, 어느 쪽이든 마무리를 합니다.

```

text = "영크크"          # 비정상
temp = 0

try:
    temp = float(text)
except ValueError:
    print("ValueError 문제가 발생했습니다")
finally:
    print(temp * 2)        # 오류가 있건 없건 실행

```

```

▶ ValueError 문제가 발생했습니다
▶ 0

```

자주 하는 실수

위 출력의 0을 눈여겨보세요. **실패했는데도** finally가 결과처럼 보이는 값을 출력합니다. finally는 "정리"에 쓰는 자리이지 "결과 출력"에 쓰는 자리가 아닙니다 — STEP 10에서 다시 다룹니다.

키워드 else 성공했을 때만 실행

정의 try에서 **예외가 하나도 발생하지 않았을 때만** 실행됩니다. 조건문의 else와 같은 단어이지만 쓰임이 다릅니다.

형태 try: 위험한 코드 except ...: 실패 시 else: 성공했을 때만 finally: 무조건

```

for text in ["62.5", "abc"]:
    try:
        temp = float(text)
    except ValueError:
        print(f"{text}: 변환 실패")
    else:
        print(f"{text}: 변환 성공 → {temp}")
    finally:
        print(" --- 한 건 처리 완료 ---")

```

```

▶ 62.5: 변환 성공 → 62.5
▶ --- 한 건 처리 완료 ---
▶ abc: 변환 실패
▶ --- 한 건 처리 완료 ---

```

왜 쓰나 try 블록을 작게 유지하는 데 도움이 됩니다. 위험한 줄만 try에 넣고, 성공한 뒤에 할 일은 else로 빼면 어디서 예외가 났는지 명확해집니다.

응용 else 없이 try 안에 성공 후 처리를 다 넣어도 동작은 같습니다. 다만 그러면 **성공 후 처리에서 난 예외까지** except가 잡아버려 **원인을 헛갈리게 만듭니다**.

```
# x try가 커서 어디서 났는지 모호
try:
    temp = float(text)
    result = 100 / temp          # 여기서 난 예외도 아래가 잡는다
except ValueError:
    print("변환 실패")

# ✓ else로 분리 — try에는 위험한 줄만
try:
    temp = float(text)
except ValueError:
    print("변환 실패")
else:
    result = 100 / temp          # 성공했을 때만
```

개념

교안이 솔직하게 적어둔 대로 **넷을 다 쓸 필요는 없습니다**. 초보 단계에서는 `finally`가 가장 유용하고, `else`는 "이런 게 있다" 정도로 알아 두었다가 `try`가 커질 때 꺼내 쓰면 됩니다.

네 블록의 실행 경로

	성공 경로	실패 경로
<code>try</code>	끝까지 실행	예외가 난 줄에서 중단
<code>except</code>	건너뛴	실행
<code>else</code>	실행	건너뛴
<code>finally</code>	실행	실행

개념

표를 세로로 읽으면 규칙이 한눈에 보입니다 — `except`와 `else`는 정확히 반대이고(둘 중 하나만 실행), `finally`만 양쪽 다 실행됩니다. 그래서 `finally`에는 "어느 쪽이든 반드시 해야 하는 일"만 넣습니다. 파일 닫기, 연결 종료, 임시 파일 삭제 같은 것들이죠.

STEP 2

리소스 정리

finally 와 with open

교안이 한 장을 따로 쓴 주제입니다 — "정리는 선택이 아니라 의무." 왜 그런지 먼저 봅시다.

위험	무슨 일이	결과
① 통로 고갈	파일을 계속 열고 닫지 않으면	운영체제가 더 이상 통로를 못 내줌
② 데이터 손실	파일은 닫는 순간 실제로 저장된다	저장 대기 중인 내용이 사라짐
③ 예외 시 위험	읽다가 예외가 나면	그 아래 close가 실행되지 않음

패턴 try - finally 로 닫기 예외가 나도 반드시 닫는다

정의 finally에 close()를 넣으면 **오류 여부와 상관없이** 파일이 닫힙니다.

```
f = open("Data/09_ict_inspection_dirty.csv", "r", encoding="utf-8")

try:
    lines = f.readlines()
    value = int("abc")          # 일부러 오류
except ValueError:
    print("변환 실패")
finally:
    f.close()                   # 오류가 나도 반드시 닫힌다
    print("파일 닫힘:", f.closed)
```

▶ 변환 실패

▶ 파일 닫힘: True

왜 쓰나 ③번 위험을 정면으로 막습니다. f.close()를 try 밖 아래쪽에 두면, 중간에 예외가 나서 그 줄까지 도달하지 못하고 **파일이 열린 채 남습니다**.

응용 f.closed는 파일이 닫혔는지 알려주는 **속성**입니다(괄호 없음). 어제 배운 3초 판별법으로 보면 "점이 있고 괄호가 없으니 속성"이죠. 확인용으로 유용합니다.

```
f = open("Data/sample.txt", "r", encoding="utf-8")
print("열린 직후:", f.closed)
f.close()
print("닫은 뒤 :", f.closed)
```

▶ 열린 직후: False

▶ 닫은 뒤 : True

개념

finally로 닫는 방식은 원리를 보여주는 형태입니다. 실전에서는 아래 with open을 씁니다.

키워드 with open – 자동 정리 finally를 직접 쓴 것과 같은 효과

정의 with open(...) as f: 블록이 끝나면 파이썬이 알아서 닫습니다. 블록 안에서 예외가 나도 빠져나가기 전에 반드시 닫습니다.

```
with open("Data/sample.txt", "r", encoding="utf-8") as f:
    data = f.readlines()
    value = int("abc")          # 예외가 나도
```

블록을 벗어나며 파일은 이미 닫혀 있다

왜 쓰나 교안 표현으로 "finally를 직접 쓴 것과 같은 효과"입니다. close()를 깜빡할 여지가 없으므로 파일 처리의 표준입니다.

응용 다만 with가 자동화해 주는 것은 닫기뿐입니다. 나머지 두 가지는 여전히 직접 챙겨야 합니다.

```
import csv

# ① 블록 밖에서 쓸 값은 블록 안에서 담아둔다
rows = []
with open("Data/09_ict_inspection_dirty.csv", "r", encoding="utf-8") as f:
    rows = list(csv.reader(f))    # 밖에서 쓰려면 여기서 담아야 한다
print("행 수:", len(rows))

# ② 파일이 없는 경우는 with가 못 막는다 → try-except 필요
try:
    with open("없는파일.csv", "r", encoding="utf-8") as f:
        data = f.read()
except FileNotFoundError:
    print("파일이 없습니다")
```

▶ 행 수: 22
▶ 파일이 없습니다

자주 하는 실수

with 블록을 벗어난 뒤에 f를 쓰면 이미 닫혀 있어 ValueError: I/O operation on closed file이 납니다. 그래서 필요한 값은 블록 안에서 변수에 담아 두어야 합니다.

세 방법 비교

방법	닫기	예외 대비	실무
f = open(); f.close()	직접 — 깜빡하기 쉬움	예외 시 도달 못 함	원리 학습용
try ... finally: f.close()	직접 — 확실함	예외에도 닫힘	with를 못 쓸 때
with open(...) as f:	자동	예외에도 닫힘	기본형

STEP 3

반복문 안에서의 예외처리

continue — 가장 실용적인 패턴

교안이 "가장 실용적인 패턴", "데이터를 두려워하지 않게 만드는 무기"라고 부른 부분입니다. 어제 CHAPTER 0에서 본 "50줄 중 10번째에서 멈춤" 문제의 정답이 여기 있습니다.

키워드 try + continue 불량 줄만 건너뛰고 계속

정의 반복문 **안쪽**에 try-except를 넣고, 실패하면 except에서 continue로 다음 회차로 넘어갑니다.

형태 for x in 목록: try: 위험한 처리 except 예외: continue

```
my_list = ["123", "456", "영크크", "32", "53"]
problems = 0

for text in my_list:
    try:
        my_number = int(text)
    except ValueError:
        problems += 1
        continue          # 이 줄만 건너뛰고 다음 줄로

    print(my_number)

print(f"{problems}개는 문제가 있어서 건너뛴")

> 123
> 456
> 32
> 53
> 1개는 문제가 있어서 건너뛴
```

왜 쓰나 교안 표현으로 "수천 줄에서 불량 줄만 쓱 빼고 나머지는 모두 처리"합니다. 5천 줄 중 불량 몇 줄만 빠지고 나머지는 정상 처리되죠. 오늘 수업 코드가 정확히 이 형태입니다.

응용 continue가 없으면 어떻게 될까요? except 블록이 끝난 뒤 **아래** print(my_number)까지 실행됩니다. 그런데 my_number는 변환에 실패했으니 이전 값이 그대로 남아 있어 **영똥한 값이** 출력됩니다.

```
my_list = ["123", "456", "영크크", "32"]

# x continue 없이
for text in my_list:
    try:
        my_number = int(text)
    except ValueError:
        print(" 문제 발생")

    print(my_number)          # ← 실패해도 여기까지 온다
```

```
▶ 123
▶ 456
▶ 문제 발생
▶ 456   ← 이전 값이 그대로 출력된다
▶ 32
```

자주 하는 실수

교안 POINT 02 — try는 반복문 안쪽에 두어야 합니다. 바깥에 두면 첫 불량에서 반복 자체가 중단되어 나머지가 전부 날아갑니다.

try 위치가 만드는 차이

같은 데이터, 다른 결과

```
# x try가 반복문 바깥 — 첫 불량에서 전체 중단
try:
    for text in my_list:
        print(int(text))
except ValueError:
    print("중단됨")
```

```
# ✓ try가 반복문 안쪽 — 그 줄만 건너뛴다
for text in my_list:
    try:
        print(int(text))
    except ValueError:
        continue
```

```
▶ 123
▶ 456
▶ 중단됨 ← "32", "53" 은 처리되지 못했다
▶
▶ 123
▶ 456
▶ 32
▶ 53
```

POINT	내용	왜
① continue 역할	이번 회차를 멈추고 다음 회차로	"이 줄 건너뛰고 다음 줄로"
② try 위치	반복문 안쪽 에 둔다	바깥이면 첫 불량에서 전체 중단
③ 불량 기록	줄 번호·이유를 모아둔다	어떤 데이터가 불량이었는지 점검 가능

개념

③이 오늘 코드에서 절반만 지켜졌습니다. `problems += 1`로 개수는 세지만 어느 줄이 왜 불량이었는지는 남기지 않았습니다. 교안이 강조한 대로 **줄 번호와 이유를 함께 모아두면** 나중에 "왜 5천 줄 중 37줄이 빠졌지?"에 답할 수 있습니다. STEP 5의 `as e`가 그 "이유"를 자동으로 만들어 줍니다.

불량 기록까지 남기는 형태

```
my_list = ["123", "456", "영크크", "32", "5.5"]
bad = []                                # 개수 대신 목록으로

for i, text in enumerate(my_list, start=1):
    try:
        my_number = int(text)
    except ValueError as e:             # STEP 5
        bad.append((i, text, str(e)))   # 줄 번호 + 원본값 + 이유
        continue
    print(my_number)

print(f"불량 {len(bad)}건")
for i, text, reason in bad:
    print(f"  {i}행 {text!r}: {reason}")
```

```
▶ 123
▶ 456
▶ 32
▶ 불량 2건
▶ 3행 '영크크': invalid literal for int() with base 10: '영크크'
▶ 5행 '5.5': invalid literal for int() with base 10: '5.5'
```

STEP 4

bare except 의 위험

좋은 예외처리 4원칙

12일차부터 계속 예고했던 주제입니다. `except` 뒤에 아무것도 안 붙이고 그냥 `except:`만 써도 동작하는데, 교안은 이것을 "**피해야 할 나쁜 습관**"이라고 못 박습니다.

오늘 수업 코드에 쓰인 형태

```
# bare except - 모든 예외를 다 잡는다
try:
    my_number = int(text)
except:                                # ← 종류를 안 적음
    problems += 1
    continue
```

위험	무슨 일이	비유
① 버그 숨김	변수 오타 같은 진짜 버그도 잡아서 못 찾게 됨	경고등을 테이프로 가리는 격
② 중단 신호까지	강제 종료 신호마저 잡아 프로그램을 못 멈춤	Ctrl+C가 안 먹는다
✓ 대안	예상 예외만 적고, 새 예외를 만날 때마다 하나씩 추가	—

같은 오타, 다른 결과

```
my_list = ["123", "영크크", "32"]

# x bare except - 오타(NameError)까지 삼킨다
for text in my_list:
    try:
        my_number = int(txet)          # 변수 오타!
    except:
        continue                      # 조용히 전부 건너뛰다
print("끝 - 아무 일도 없었던 것처럼 보인다")

# ✓ 구체적으로 - 진짜 버그는 드러난다
for text in my_list:
    try:
        my_number = int(txet)
    except ValueError:
        continue

# NameError: name 'txet' is not defined ← 제대로 터진다
```

▶ 끝 - 아무 일도 없었던 것처럼 보인다

▶

▶ NameError: name 'txet' is not defined

자주 하는 실수

위 예시가 무서운 이유는 **결과가 그럴듯하게 나오기 때문**입니다. bare except 버전은 "처리 완료"처럼 보이지만 실제로는 **한 건도 처리하지 못했습니다**. 어제 배운 세 층 중 "**결과 오류**"로 떨어지는 것이죠. 오늘 수업 코드의 except:는 교안을 따라 한 것이라 그대로 두어도 되지만, **앞으로 새로 쓸 때는 반드시 종류를 적으세요**.

좋은 예외처리 4원칙

원칙	내용	실천
① 구체적으로	bare except 대신 예상되는 예외만 정확히 지정	<code>except ValueError:</code>
② 작게	위험한 코드만 try에 담아 오류 위치를 명확히	변환하는 한 줄만
③ 기록	어떤 데이터가 왜 빠졌는지 남겨야 분석 신뢰도가 오른다	<code>bad.append((id, str(e)))</code>
④ 드러내기	예상 못한 예외는 잡지 말고 드러내서 코드를 고치게 둔다	넓은 except를 남발하지 않기

개념

교안이 한 문장으로 정리한 것이 핵심입니다 — "예상한 문제는 우아하게 처리하고, 예상 못한 문제는 숨기지 말고 드러내라." 이 균형 감각이 견고한 프로그램과 위험한 프로그램을 가릅니다. 모든 것을 잡으면 안전해 보이지만 실제로는 무엇이 잘못됐는지 영원히 모르게 됩니다.

구분	나쁜 예	좋은 예
예외 지정	<code>except: # 전부 잡기</code>	<code>except ValueError:</code>
try 범위	전체를 감쌌	위험한 줄만
문제 처리	무시 (<code>pass</code> · <code>continue</code> 만)	줄 번호 · 이유 기록

STEP 5

as e — 에러 정보 꺼내 쓰기

무엇이 왜 실패했는지 기록

문법 `except ... as e` 예외 객체를 변수에 담기

정의 `except` 뒤에 `as e`를 붙이면 **예외 정보**가 `e`에 담깁니다. 출력하면 트레이스백 맨 아랫줄에서 봤던 그 구체적인 설명이 나옵니다.

형태 `except ValueError as e: print(e)`

```
for text in ["62.5", "영크크", ""]:
    try:
        temp = float(text)
        print(f"{text!r} → {temp}")
    except ValueError as e:
        print(f"{text!r} → 실패: {e}")
```

- ▶ '62.5' → 62.5
- ▶ '영크크' → 실패: could not convert string to float: '영크크'
- ▶ '' → 실패: could not convert string to float: ''

왜 쓰나 단순히 "문제 발생"이 아니라 **어떤 값이 왜 문제인지**를 기록할 수 있습니다. STEP 4의 원칙 ③(기록)을 실현하는 도구죠.

응용 교안이 든 세 가지 활용 관점이 실무 그대로입니다 — ① **맥락 결합**: 내가 아는 줄 번호와 파이썬이 주는 원인을 합쳐 더 정확한 기록을 만들고, ② **사용자용**: 화면에는 이해하기 쉬운 우리말 안내를, ③ **개발자용**: 기록에는 `e`의 구체적 원인을 남깁니다.

```
import csv

bad = []
with open("Data/09_ict_inspection_dirty.csv", "r", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        check_id = row.get("검사ID", "?")
        try:
            value = float(row["측정값"])
        except (ValueError, TypeError) as e:
            bad.append((check_id, str(e)))    # ③ 개발자용 기록
            print(" 측정값을 읽을 수 없습니다.") # ② 사용자용 안내
            continue

print(f"\n[불량 목록] {len(bad)}건")
for cid, reason in bad[:3]:
    print(f" 검사ID {cid}: {reason}")
```

- ▶ [불량 목록] 4건
- ▶ 검사ID 9001: could not convert string to float: '결측'
- ▶ 검사ID 9002: could not convert string to float: ''
- ▶ 검사ID 9004: float() argument must be a string or a real number, not 'NoneType'

개념

e는 문자열이 아니라 **예외 객체**입니다. 그래서 리스트에 담아 나중에 파일로 쓰려면 str(e)로 바꿔야 안전합니다. print(e)는 파이썬이 알아서 문자열로 바꿔 주므로 그냥 써도 됩니다.

as e 가 있고 없과의 차이

	as e 없이	as e 로
기록되는 것	"문제 발생" — 어느 줄인지만	무엇이 왜 실패했는지
나중에 할 수 있는 일	개수 세기	원인별 분류 · 재처리 · 데이터 수정 요청
리포트	"불량 13건"	"결측 4건 · 범위 초과 9건"

설비 적용

현장에서는 "왜 빠졌는지"가 곧 **개선 과제**가 됩니다. 결측이 많으면 센서 통신을 점검해야 하고, 범위 초과가 많으면 기준값 설정을 다시 봐야 합니다. as e로 남긴 한 줄이 그 판단의 근거가 됩니다. STEP 9에서 만들 리포트의 [불량 목록] 섹션이 정확히 이 기록으로 채워집니다.

STEP 6

raise — 규칙을 강제하기

파이썬이 못 잡는 우리만의 기준

여기가 오늘 개념 중 가장 새로운 부분입니다. 지금까지는 **파이썬이 일으킨 예외를 받아냈다면**, **raise**는 **내가 예외를 직접 일으킵니다**.

키워드 **raise** 직접 예외를 발생시킨다

정의 **raise** 예외종류 ("메시지")로 예외를 일으킵니다. 이렇게 일으킨 예외도 **try-except**로 잡을 수 있습니다.

형태 **raise** **ValueError**("메시지")

```
temp = -999

if temp > 200:
    raise ValueError("비정상")

# 문제: -999는 파이썬 입장에서 완전히 정상적인 숫자다
print(float("-999"))          # 아무 예외도 안 난다
> -999.0
```

왜 쓰나 교안 설명이 정확합니다 — "영하 999도 같은 값은 파이썬 입장에선 정상 숫자입니다. 아무 예외도 나지 않습니다. 하지만 우리는 이 값이 센서 오류인 것을 알고 있습니다." 파이썬은 문법만 보지 **현실의 규칙**은 모릅니다.

응용 그래서 우리가 정한 범위를 벗어나면 **raise**로 직접 예외를 일으킵니다. 그러면 **형변환 실패**와 **범위 위반**을 같은 **except**에서 함께 처리할 수 있습니다 — 둘 다 "이 줄은 못 쓴다"는 같은 결론이니까요.

```

values = ["62.5", "영크크", "-999", "180"]

for v in values:
    try:
        temp = float(v)                                # ① 파이썬이 잡는 오류
        if temp < -50 or temp > 200:
            raise ValueError(f"{temp}도는 정상 범위(-50~200)를 벗어남")    # ② 내가 만드는
오류
    except ValueError as e:
        print(f" 건너뛴 — {e}")
        continue
    print(f"정상: {temp}도")

```

- ▶ 정상: 62.5도
- ▶ 건너뛴 — could not convert string to float: '영크크'
- ▶ 건너뛴 — -999.0도는 정상 범위(-50~200)를 벗어남
- ▶ 정상: 180.0도

개념

교안 POINT 03 — 값이 부적절한 경우이니 ValueError를 쓰는 것이 자연스럽습니다. 예외 종류는 아무거나 쓰는 게 아니라 의미에 맞게 고릅니다.

raise 활용 세 가지 포인트

POINT	내용	실무에서는
① 현실 규칙	온도·압력·진동마다 현실적인 정상 범위가 있다	raise로 그 범위를 강제
② 역할 분리	검증 함수는 raise로 예외만 일으킨다	대응은 호출하는 쪽이 try-except로
③ 예외 종류	값이 부적절한 경우	ValueError

② 역할 분리 — 검증 함수와 대응의 분리

```
def check_temp(value):  
    """온도를 검증한다. 범위를 벗어나면 ValueError를 일으킨다."""  
  
    temp = float(value)                # 변환 실패도 ValueError  
    if temp < -50 or temp > 200:  
        raise ValueError(f"{temp}도는 정상 범위를 벗어남")  
  
    return temp                        # 검증 통과한 값만 돌려준다  
  
# 호출하는 쪽이 "어떻게 대응할지"를 정한다  
for v in ["62.5", "-999", "영크크"]:  
    try:  
        print("정상:", check_temp(v))  
    except ValueError as e:  
        print("  제외:", e)
```

- ▶ 정상: 62.5
- ▶ 제외: -999.0도는 정상 범위를 벗어남
- ▶ 제외: could not convert string to float: '영크크'

연결

11일차에 배운 "함수 하나에 기능 하나"가 여기서 다시 나옵니다. `check_temp`는 **검증만** 하고, 불량일 때 건너뛸지 기본값을 넣을지 로그를 남길지는 **호출하는 쪽**이 정합니다. 그래서 같은 검증 함수를 다른 상황에서 다르게 쓸 수 있죠. `raise`가 그 분리를 가능하게 하는 장치입니다.

세 가지 예외의 출처

출처	예	누가 일으키나
문법·연산 실패	<code>float("영크크")</code> → <code>ValueError</code>	파이썬
환경 문제	<code>open("없는파일")</code> → <code>FileNotFoundError</code>	파이썬 (운영체제 경유)
현실 규칙 위반	<code>temp > 200</code> → <code>raise ValueError</code>	내가 직접

STEP 7

데이터 처리 파이프라인 설계

다섯 단계 · 나누는 이유

여기부터 교안 09_03입니다. 교안 첫 장의 선언이 이 단원의 성격을 말해줍니다 — "새 문법은 없다. 이미 다 배운 부품을 조립해, 작동하는 한 대의 기계를 만드는 것이다."

	내용
만들 프로그램	설비 센서 CSV를 읽어 정리하고, 불량률 걸러내고, 통계를 내고, 리포트로 저장한다
쓰이는 문법	변수 · 조건문 · 반복문 · 함수 · 파일 입출력 · 예외처리 — 지금까지 배운 전부
성격	한 번에 완성하지 않는다. 단계로 쌓고, 막힐 때마다 어느 단계인지 안다
의미	완성된 분석 프로그램은 그 자체로 포트폴리오 자산이 된다

파이프라인이란

앞 단계 결과가 뒤 단계의 재료가 되는 연결 구조를 **파이프라인(pipeline)**이라 부릅니다. 교안의 비유는 수도관입니다 — 수도관을 따라 물이 정수되듯, 지저분한 원본 데이터가 파이프라인을 거치며 분석 가능한 결과로 바뀝니다.



STEP	하는 일	쓰이는 문법
01 읽기	CSV 파일을 열고 헤더와 데이터를 분리 · 파일 없음에 대비	파일 입출력 · 예외처리
02 분류	부품별로 묶기	조건문 · 반복문 · 딕셔너리
03 계산	평균 · 최대 · 최소를 함수로 만들기	함수 · 반복문
04 방어	불량 데이터 거르기	예외처리 총동원 (continue · raise · as e)
05 저장	리포트로 남기기	파일 쓰기 · f-string

왜 단계로 나누는가

이유	내용
① 단계별 문법	읽기는 파일, 분류는 딕셔너리, 방어는 예외처리 — 배운 문법이 단계마다 한 번씩 등장한다

이유	내용
② 독립 검증	각 단계를 따로 만들고 먼저 테스트한 뒤 붙인다 → 오류 난 단계를 찾기 쉬워진다
③ 부분 수정	분석 항목이 늘어나도 계산 단계만 손보면 나머지는 그대로 둔다

개념

교안이 덧붙인 한 줄이 중요합니다 — **"NumPy·Pandas도 결국 이 단계를 빠르게 처리하는 도구."** 지금 손으로 만드는 다섯 단계가 사라지는 게 아니라, 각 단계가 한 줄로 짧아질 뿐입니다. **구조를 손으로 한 번 만들어 봐야** 나중에 라이브러리가 무엇을 대신해 주는지 이해할 수 있습니다.

연결

강사님이 오전 수업에서 강조하신 방법이 있습니다 — **"한 코드가 쪽 나열돼 있더라도 무작정 짜는 게 아니라, 단계별이라는 생각을 가지고 일단 주석을 만드는 것."** 실제로 수업 코드 10_student_scores에도 # 0. 준비 → # 1. 파일을 연다 → # 2. 리스트 데이터를 얻는다 → # 3. 점수 계산 → # 4. 결과를 보여주기가 주석으로 먼저 깔려 있습니다. **주석으로 뼈대를 잡고 살을 붙이는** 방식이 파이프라인 사고의 시작입니다.

STEP 8

파이프라인 1~3단계

읽기 · 분류 · 통계

1단계 — CSV 읽기

목표는 "센서 CSV를 안전하게 열어 헤더와 데이터를 분리"하고, "파일이 없어도 멈추지 않게 FileNotFoundError에 대비"하는 것입니다. 그리고 함수로 만들어 이후 단계에서 재사용합니다.

1단계 — 읽기

```
import os, csv
```

```
def read_csv(file_path):
    """CSV를 읽어 (헤더, 데이터행) 을 돌려준다. 파일이 없으면 빈 결과를 돌려준다."""
    try:
        with open(file_path, "r", encoding="utf-8") as f:
            reader = csv.reader(f)
            header = next(reader)
            rows = list(reader)
            return header, rows
    except FileNotFoundError:
        print(f"파일을 찾을 수 없습니다: {file_path}")
        return [], []          # 빈 결과를 돌려줘 다음 단계가 멈추지 않게
```

```
path = os.path.join("Data", "09_ict_inspection_dirty.csv")
header, rows = read_csv(path)
print("헤더:", header)
print("데이터 행 수:", len(rows))
```

- ▶ 헤더: ['검사ID', '부품명', '측정값', '기준값', '상한치', '하한치', '검사결과']
- ▶ 데이터 행 수: 21

개념

예외가 났을 때 빈 결과([], [])를 돌려주는 것이 핵심입니다. None을 돌려주면 다음 단계에서 len(None)으로 또 터집니다. 빈 리스트는 반복해도 0번 돌 뿐 오류가 나지 않으므로, 파이프라인이 조용히 흘러갑니다. 11일차에 배운 "함수는 return으로 소통한다"가 여기서 실전으로 쓰입니다.

2단계 — 조건 분류

읽어온 데이터를 **부품별로 나누어 딕셔너리에 담습니다**. 키는 부품명, 값은 행 리스트입니다. 교안이 짙은 핵심 패턴은 **"처음 보는 부품명이면 빈 리스트를 먼저 만들고, 거기에 행을 추가"**하는 것입니다.

2단계 — 분류

```
import csv

def group_by_part(rows):
    """행 목록을 부품명별로 묶어 딕셔너리로 돌려준다."""
    grouped = {}

    for row in rows:
        part = row["부품명"]

        if part not in grouped:      # 처음 보는 부품명이면
            grouped[part] = []      # 빈 리스트부터 만들고
            grouped[part].append(row) # 거기에 행을 추가

    return grouped

with open(path, "r", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))

grouped = group_by_part(rows)
for part, items in grouped.items():
    print(f"{part}: {len(items)}개")
```

- ▶ E2: 4개
- ▶ F2: 4개
- ▶ B2: 4개
- ▶ A2: 4개
- ▶ D2: 3개
- ▶ : 2개 ← 부품명이 빈 행

주의

출력 마지막의 빈 키(: 2개)를 보세요. -2,,,,,0K 같은 행은 부품명이 빈 문자열이라 "" 라는 키가 만들어집니다. 오류는 안 나지만 결과에 의미 없는 항목이 생기죠. 1단계에서 걸러도 되고, 4단계 방어에서 걸러도 됩니다. 어느 단계에서 거를지 정하는 것도 파이프라인 설계의 일부입니다.

3단계 — 통계 함수

평균·최대·최소를 계산하는 함수를 만듭니다. 교안이 주의점 둘을 명시했습니다 — ① 숫자가 아닌 값은 건너뛰다 ② 값이 하나도 없으면 None을 반환해 0으로 나누는 오류를 막는다.

3단계 — 통계 함수

```
def calc_stat(rows, col):
    """행 목록에서 col 열의 (개수, 평균, 최소, 최대)를 돌려준다.

    숫자가 아닌 값은 건너뛰다. 유효한 값이 하나도 없으면 None을 돌려준다.
    """
    total = 0
    count = 0
    min_val = None
    max_val = None

    for row in rows:
        try:
            value = float(row[col])
        except (ValueError, TypeError, KeyError):
            continue                # ① 숫자가 아니면 건너뛰다

        total += value
        count += 1
        if min_val is None or value < min_val:
            min_val = value
        if max_val is None or value > max_val:
            max_val = value

    if count == 0:
        return None                # ② 0으로 나누기 방지

    return count, total / count, min_val, max_val
```

개념

`min_val = None`으로 시작해 `if min_val is None or value < min_val`로 갱신하는 방식을
눈여겨보세요. 9일차 종합 실습에서 `min_val = 0`으로 시작해 최솟값이 영원히 0이었던 버그를 기억하실 겁니다.
None으로 시작하면 첫 값이 무조건 들어가므로 음수든 0이든 정확합니다. `float("inf")`를 쓰는 방법과 함께
알아두면 좋은 두 번째 정석입니다.

파이프라인 4~6단계

방어 · 리포트 · 검증

4단계 — 불량 방어

교안이 "앞서 배운 모든 예외처리 기법을 총동원"한다고 예고한 단계입니다. 실제로 `try-except` · `continue` · `raise` · `as e`가 한 함수에 모두 들어갑니다.

4단계 — 방어

```
import csv

def filter_valid(file_path):
    """정상 행은 valid에, 불량 행은 (검사ID, 사유)로 bad에 담아 돌려준다."""
    valid = []
    bad = []

    with open(file_path, "r", encoding="utf-8") as f:
        for row in csv.DictReader(f):
            check_id = row.get("검사ID", "?")
            try:
                value = float(row["측정값"])          # ① 파이썬이 잡는 오류
                low = float(row["하한치"])
                high = float(row["상한치"])

                if value < low or value > high:
                    raise ValueError(                # ② 내가 만드는 오류
                        f"측정값 {value}이(가) 정상범위({low}~{high})를 벗어남")

                valid.append(row)

            except (ValueError, TypeError, KeyError) as e:  # ③ 한 곳에서 함께
                bad.append((check_id, str(e)))              # ④ 이유까지 기록
                continue

    return valid, bad

valid, bad = filter_valid(path)
```

```
print(f"정상 {len(valid)}개 / 불량 {len(bad)}개")
```

▶ 정상 8개 / 불량 13개

기법	이 함수에서 맡는 일	STEP
try-except	숫자로 못 바꾸는 값 · 열이 없는 행을 받아냄	13일차
raise	범위를 벗어난 값을 직접 예외로 만들	STEP 6
as e	무엇이 왜 실패했는지 사유를 뽑아냄	STEP 5
continue	불량 줄만 건너뛰고 다음 행으로	STEP 3

개념

가장 영리한 부분은 raise한 예외를 같은 except가 받는다는 점입니다. **형변환 실패든 범위 초과든 결론은 같습니다** — "이 줄은 못 쓴다." 그래서 처리도 한 곳에서 하고, 차이는 as e가 뽑은 사유 문자열로만 남습니다. 그 덕분에 리포트에서 could not convert...와 정상범위를 벗어남이 나란히 보이죠.

5단계 — 리포트 저장

교안이 리포트 원칙 셋을 제시합니다 — ① **핵심 먼저** ② **단위 붙이기** ③ **구분선 활용**. 그리고 작성 방식은 "리포트 줄들을 리스트에 차곡차곡 모은 뒤, 마지막에 반복문으로 한 번에 파일에 쓰기"입니다.

원칙	내용	왜
① 핵심 먼저	전체 · 정상 · 불량 요약은 맨 위에	읽는 사람이 가장 먼저 알고 싶은 것
② 단위 붙이기	숫자마다 "도" · "개" 같은 단위와 설명을 함께	"65.4"만 적으면 무슨 값인지 모름
③ 구분선 활용	줄을 나누고 구분선으로 영역을 가른다	요약과 세부를 눈으로 분리

5단계 — 리포트 저장

```
def save_report(file_path, output_path):
    """분석 결과를 리포트 형식으로 정리해 txt 파일에 저장한다."""
    valid, bad = filter_valid(file_path)          # 4단계 재사용
    stat = calc_stat(valid, "측정값")             # 3단계 재사용

    lines = []                                     # 줄을 리스트에 모은다
    lines.append("=== ICT 검사 데이터 분석 리포트 ===")
    lines.append(f"전체 {len(valid) + len(bad)}행 · 정상 {len(valid)}개 · 불량
```

```
{len(bad)}개")
    lines.append("-" * 40)

    if stat is None:
        lines.append("측정값: 유효한 숫자 데이터 없음")
    else:
        _, avg, min_val, max_val = stat
        lines.append(f"측정값 평균 — {avg:.2f}")
        lines.append(f"측정값 최고 — {max_val}")
        lines.append(f"측정값 최저 — {min_val}")

    lines.append("-" * 40)
    lines.append("[불량 목록]")
    for check_id, reason in bad:
        lines.append(f"검사ID {check_id}: {reason}")

    with open(output_path, "w", encoding="utf-8") as f:
        for line in lines:
            # 마지막에 한 번에 쓴다
            f.write(line + "\n")
```

실제 저장된 리포트 (Data/09_ict_inspection_report.txt)

=== ICT 검사 데이터 분석 리포트 ===

전체 21행 · 정상 8개 · 불량 13개

측정값 평균 — 673.16

측정값 최고 — 3430.02

측정값 최저 — 0.0

[불량 목록]

검사ID 9001: could not convert string to float: '결측'

검사ID 2203: 측정값 2939.61이(가) 정상범위(-3750.0~2310.0)를 벗어남

검사ID 9004: float() argument must be a string or a real number, not 'NoneType'

...

개념

줄을 리스트에 모았다가 마지막에 한 번에 쓰는 방식이 왜 좋을까요? ① 파일을 여는 시간이 짧아 안전하고, ② 저장 전에 `print(lines)`로 내용을 미리 확인할 수 있고, ③ 나중에 화면 출력과 파일 저장을 **같은 데이터로** 할 수 있습니다. 중간에 오류가 나도 파일이 반쯤 쓰인 채 남지 않는다는 점도 큼니다.

6단계 — 통계 검증

마지막 단계는 **결과가 맞는지 스스로 확인**하는 것입니다. 2단계 `딕셔너리`와 3단계 `함수`를 재사용해 부품별 통계를 내고, 부품별 개수의 합이 전체 정상 개수와 같은지 봅니다.

6단계 — 검증

```
def verify_by_part(file_path):
    """부품별 통계를 내고, 개수 합이 전체 정상 개수와 맞는지 검증한다."""
    valid, _ = filter_valid(file_path)          # 4단계 재사용
    grouped = group_by_part(valid)              # 2단계 재사용

    total_by_part = 0
    for part, rows in grouped.items():
        result = calc_stat(rows, "측정값")       # 3단계 재사용
        if result is None:
            print(f"{part}: 유효 데이터 없음")
            continue
        count, avg, min_val, max_val = result
        total_by_part += count
        print(f"{part}: {count}개 | 평균 {avg:.2f}")

    print("-" * 40)
    print(f"부품별 개수 합계: {total_by_part}")
    print(f"전체 정상 개수: {len(valid)}")
    print(f"검증 결과: {total_by_part == len(valid)}")
```

```
▶ E2: 3개 | 평균 0.00
▶ F2: 1개 | 평균 0.00
▶ B2: 2개 | 평균 0.00
▶ A2: 2개 | 평균 1477.19
▶ -----
▶ 부품별 개수 합계: 8
▶ 전체 정상 개수: 8
▶ 검증 결과: True
```

개념

교안이 검증하는 이유를 이렇게 설명합니다 — **"숫자가 맞아떨어지면, 파이프라인이 데이터를 빠뜨리거나 중복 없이 정확히 처리했다는 증거다."** 이걸 11일차 함수 설계 체크리스트의 **"여러 입력으로 테스트했는가"**를 파이프라인 규모로 확장한 것입니다. 그리고 이 단계에서 **1~4단계 함수를 전부 재사용**한다는 점이 중요합니다 — 함수로 나눠 놓았기 때문에 가능한 일이죠.

여섯 단계 재사용 구조

단계	함수	재사용하는 곳
1 읽기	<code>read_csv()</code>	2단계 입력
2 분류	<code>group_by_part()</code>	6단계에서 재사용
3 통계	<code>calc_stat()</code>	5·6단계에서 재사용
4 방어	<code>filter_valid()</code>	5·6단계에서 재사용
5 저장	<code>save_report()</code>	3·4단계를 불러 씀
6 검증	<code>verify_by_part()</code>	2·3·4단계를 불러 씀

연결

오른쪽 열을 보면 **3·4단계 함수가 각각 두 번씩 재사용**됩니다. 11일차에 배운 "**작은 함수들의 조합이 프로그래밍의 힘**"이 실제로 작동하는 모습이죠. 만약 통계 계산을 5단계와 6단계에 각각 복사해 넣었다면, 계산식을 고칠 때 두 곳을 고쳐야 하고 한 곳을 빠뜨리면 결과가 어긋납니다. **함수로 나눈 덕분에 고칠 곳이 한 군데**입니다.

STEP 10

내 코드 점검 — 수업 코드 4개

도달 불가 except · 이름 불일치

오늘 배포된 수업 코드는 네 개입니다. 두 개는 09_02(예외처리), 두 개는 09_03 도입부(학생 점수 집계)입니다. 강사님이 시연하신 코드라 큰 문제는 없지만, **앞 회차에서 짚었던 것과 같은 유형**이 몇 군데 있습니다.

09_finally — except NameError 는 도달할 수 없습니다

수업 코드

```
text = "영크크"
temp = 0

try:
    temp = float(text)
except ValueError:
    print("ValueError 문제가 발생했습니다")
except NameError:
    # ← 도달 불가
    print("NameError 문제가 발생했습니다")
finally:
    print(temp * 2)
```

▶ ValueError 문제가 발생했습니다
▶ 0

	왜 도달 불가인가
NameError	정의되지 않은 이름을 썼을 때 나는 예외입니다
이 코드에서는	text가 바로 위에 정의돼 있으므로 NameError가 날 수 없습니다
float()에서 나는 예외	ValueError · TypeError
여기서 함께 쓸 만한 것	except TypeError: # float(None) 같은 경우

개념

13일차 20_handler.py의 except TypeError, 12일차 except: 와 같은 유형입니다. 예외 종류를 여러 개 나열하는 연습 자체는 좋은데, **그 자리에서 실제로 날 수 있는 예외인지** 한 번 따져보는 습관을 붙이면 좋습니다. 방법은 간단합니다 — try 안의 코드를 그대로 실행해 보고 무슨 예외가 나는지 확인하면 됩니다.

09_finally — finally에서 결과를 출력하고 있습니다

출력의 마지막 0을 보세요. **변환에 실패했는데도** `temp * 2 = 0`이 결과처럼 출력됩니다. `finally`는 "정리"의 자리이지 "결과 발표"의 자리가 아닙니다.

else를 살린 형태

```
# x 실패해도 결과처럼 보인다
finally:
    print(temp * 2)          # 실패 시 0 → "결과가 0인가?" 오해

# ✓ 성공했을 때만 결과를 낸다 (else 활용)
try:
    temp = float(text)
except ValueError:
    print("ValueError 문제가 발생했습니다")
else:
    print(temp * 2)          # 성공했을 때만
finally:
    print("--- 처리 종료 ---") # 정리만
```

▶ `ValueError` 문제가 발생했습니다

▶ --- 처리 종료 ---

연결

교안 09_02 p4가 `else`와 `finally`를 한 장에 나란히 소개한 이유가 이것입니다 — **성공 결과는 else**, 마무리는 `finally`. 수업 코드에는 `else`가 빠져 있어 둘의 역할이 겹쳐 보입니다. STEP 1의 실행 경로 표를 다시 보면 왜 나눠야 하는지 분명해집니다.

09_continue — bare except 입니다

`except:` 뒤에 종류가 없습니다. 같은 교안 09_02 p18~19가 "**피해야 할 나쁜 습관**"이라고 지목한 바로 그 형태입니다.

고치는 방향

```
try:
    my_number = int(text)
except:
    problems += 1
    continue

# ✓ 구체적으로 + 사유까지 기록
except ValueError as e:
    bad.append((text, str(e)))
    continue
```

주의

교안 안에서 배우는 내용과 예제 코드가 어긋난 경우입니다. p18에서 bare except를 금지하면서 예제는 bare except를 쓰고 있죠. 아마 continue 패턴 자체에 집중시키려고 단순화한 것으로 보입니다. 개념을 보여줄 때는 단순하게, 실제로 쓸 때는 구체적으로 — 그렇게 구분해 받아들이면 됩니다. 그리고 `problems += 1`은 개수만 세는데, STEP 3에서 본 것처럼 줄 번호와 사유를 함께 모으는 편이 나중에 훨씬 쓸모 있습니다.

10_student_scores — total 이라는 이름이 실제로는 평균입니다

수업 코드

주석: "각 학생별 합계와 모든 학생들의 평균 점수를 내는 코드"

```
total = (kor + eng + math) / 3          # ← 3으로 나눴으니 평균이다
print(f"{name} | {kor} | {eng} | {math} | {total}")
```

```
students_count += 1
total_all += total                      # 평균들의 합
```

```
avg_all = total_all / students_count    # 평균의 평균
```

```
▶ 김민준 | 90 | 57 | 51 | 66.0
▶ ...
▶ 전체 20명 | 평균 72.88333333333333점
```

	코드	실제 값	이름이 말하는 것
합계	<code>kor + eng + math</code>	198	—
코드의 total	<code>(kor + eng + math) / 3</code>	66.0	합계라고 읽히지만 평균
출력 라벨	plus 버전의 "합계점수"	89.67	역시 평균

자주 하는 실수

11일차에 배운 "이름과 하는 일이 일치해야 한다"가 그대로 걸립니다. total은 관례상 합계를 뜻하는데 값은 평균이죠. 실행에는 문제가 없지만 읽는 사람이 오해하고, 나중에 "진짜 합계"가 필요해지면 이름 충돌이 납니다. student_avg처럼 바꾸면 코드가 스스로 설명합니다 — 제출하신 09_02_example.py에서는 실제로 student_avg로 고쳐 쓰셨습니다.

10_student_scores — BOM 과 예외처리

수업 코드의 두 지점

```
name = row.get("\ufeff이름", "(이름없음)")    # ← BOM을 이름에 붙여 씀
kor = int(row.get("국어", "0"))              # ← 예외처리 없음
```

고친 형태

```
# ① BOM — encoding="utf-8-sig" 로 열면 자동으로 제거된다
with open(file_path, "r", encoding="utf-8-sig") as f:
    reader = csv.DictReader(f)
    for row in reader:
        name = row.get("이름", "(이름없음)")    # 깔끔하게

# ② 오늘 배운 예외처리를 없으면
try:
    kor = int(row["국어"])
    eng = int(row["영어"])
    math = int(row["수학"])
except (ValueError, KeyError, TypeError) as e:
    print(f" {name}: 점수를 읽을 수 없어 건너뛰니다 — {e}")
    continue
```

```
▶ 김민준 | 90 | 57 | 51 | 평균 66.0
▶ ...
```

개념

BOM(Byte Order Mark)은 파일 맨 앞에 붙는 보이지 않는 3바이트(EF BB BF)입니다. 윈도우 엑셀이 CSV를 저장할 때 붙이는데, encoding="utf-8"로 열면 **첫 열 이름 앞에** \ufeff가 그대로 붙어 row["이름"]이 None이 됩니다. utf-8-sig로 열면 파이썬이 알아서 떼어냅니다. 12일차에 배운 인코딩 이야기의 실전판이죠. 그리고 예외처리는 — **강사님도 수업 중에** "학생들 점수 코드엔 예외처리가 없었죠? 왜 없었겠어요?"라고 짚으셨습니다. 깨끗한 데이터라 없어도 됐지만, dirty 데이터에는 반드시 필요합니다.

STEP 11

실습 점검 — 09_02 · 09_03

파이프라인 완주 확인

제출하신 실습 두 개를 확인했습니다. 결론부터 말하면 **09_03 파이프라인은 6단계를 모두 완주했고 검증까지 통과**했습니다. 오늘 배운 예외처리 기법 네 가지가 정확한 자리에 들어가 있습니다.

파일	내용	상태
Practice/09_02_example.py	학생 점수 집계 + 과목별 최고점 (제출 안 하는 실습 포함)	동작 정확 · 개선점 3건
Practice/09_03_example.py	파이프라인 1~6단계 전부	완주 · 검증 True
Data/ 09_ict_inspection_report.txt	5단계 리포트 산출물	생성 확인

09_03 — 잘한 부분

항목	제출 코드에서	교안 원칙
구체적 예외	<code>except (ValueError, TypeError, KeyError) as e:</code>	4원칙 ① 구체적으로
사유 기록	<code>bad.append((check_id, str(e)))</code>	4원칙 ③ 기록
raise 활용	범위 위반을 직접 예외로	STEP 6 그대로
0 나누기 방지	<code>if count == 0: return None</code>	3단계 주의점
None 초기화	<code>min_val = None</code> → <code>is None or <</code>	9일차 버그의 정석 해법
함수 재사용	<code>group_by_part(file_path, rows=valid)</code>	6단계에서 2·3·4단계 재사용

개념

특히 `min_val = None`으로 시작해 `if min_val is None or value < min_val`로 갱신한 부분이 좋습니다. 9일차 종합 실습에서 `min_val = 0`으로 시작해 최솟값이 영원히 0이었던 버그를 정확히 피한 형태입니다. 그리고 `rows=valid`를 넘겨 파일을 다시 읽지 않게 한 설계도 눈에 띕니다 — 매개변수 기본값 (`rows=None`)을 활용해 "파일에서 읽거나, 넘겨받은 행을 쓰거나" 두 방식을 한 함수로 처리했습니다. 11일차 기본값 인자의 실전 활용입니다.

09_03 — 다음을 부분 세 가지

① 1단계가 값을 돌려주지 않습니다. 교안 p9는 "함수로 만들어 이후 단계에서 그대로 재사용한다"고 했는데, `print_col()`은 화면에 출력만 하고 `return`이 없습니다. 그래서 2단계가 파일을 다시 읽습니다.

① 1단계 — `return` 추가

```
# 현재 — 출력만 하고 끝
def print_col(file_path):
    ...
    print(f"총 데이터 개수: {count}")
    # return 이 없다 → 2단계가 파일을 다시 연다

# 고치면 — 결과를 돌려줘 다음 단계가 이어받는다
def read_csv(file_path):
    """CSV를 읽어 (헤더, 데이터행)을 돌려준다."""
    try:
        with open(file_path, "r", encoding="utf-8") as f:
            reader = csv.reader(f)
            header = next(reader)
            rows = list(reader)
            return header, rows
    except FileNotFoundError:
        print(f"파일 없음: {file_path}")
        return [], []
```

연결

11일차 STEP 7의 "**print 하는 함수 vs return 하는 함수**"가 여기서 다시 나옵니다. 화면에 보여주는 것과 값을 넘기는 것은 다른 일이죠. 함수 이름도 `print_col`(열을 출력)보다 `read_csv`가 정확합니다 — 실제로 하는 일은 **열이 아니라 행을 읽는 것이니까요.**

② `calc_statistic`이 모든 열에 통계를 냅니다. **검사ID·기준값처럼 평균이 의미 없는 열까지 계산합니다.** 숫자라서 오류는 안 나지만 읽는 사람이 오해할 수 있습니다.

② 분석 대상 열 지정

현재 — 모든 열

```
for col in header:
    result = calc_stat(rows, col)
```

고치면 — 분석할 열만 지정

```
TARGET_COLS = ["측정값"] # 필요하면 여기만 늘린다
for col in TARGET_COLS:
    result = calc_stat(rows, col)
```

▶ 검사ID: 유효 데이터 21개 | 평균: 3105.05 ← 의미 없는 통계

▶ 측정값: 유효 데이터 17개 | 평균: 1462.34

③ 부품명이 빈 행이 그대로 그룹에 남습니다. -2,,,,,0K 같은 행 때문에 "" 라는 키가 생깁니다. 4단계에서 걸러지므로 최종 결과에는 영향이 없지만, 2단계 단독 출력에는 나타납니다.

③ 빈 키 방지

```
for row in rows:
    part = row.get("부품명", "")
    if not part: # 빈 부품명은 건너뛴다
        continue
    if part not in grouped:
        grouped[part] = []
    grouped[part].append(row)
```

09_02 — 학생 점수 실습

수업 코드의 total 이름 문제를 student_avg로 정확히 고쳐 쓰셨고, :.1f 서식으로 소수점도 정리하셨습니다. 남은 것은 세 가지입니다.

항목	현재	권장
BOM 처리	row.get("\uffeff이름", 0)	encoding="utf-8-sig" + row.get("이름", "")
기본값 타입	이름 기본값이 숫자 0	문자열 자리이므로 ""
코드 중복	같은 읽기·집계 블록이 두 번	함수로 묶어 재사용 (11일차)

응용

두 블록이 파일 열기·헤더 처리·평균 계산까지 거의 같습니다. 11일차의 "찾기 → 묶기 → 교체 → 확인" 절차대로 공통부를 read_scores() 함수로 묶고 달라지는 부분만 뒤에 붙이면, 09_03의 파이프라인과 같은 모양이 됩니다.

치트시트

견고한 예외처리와 파이프라인

예외처리 네 블록

블록	언제 실행	쓰는 것
try:	항상 — 위험한 코드를 담는다	변환·파일 열기 등 실패 가능한 줄만
except 예외:	그 예외가 났을 때	안내·기록·continue
else:	예외가 하나도 안 났을 때	성공 후 처리·결과 출력
finally:	성공·실패 무조건	파일 닫기·마무리 (결과 출력은 X)

오늘의 문법 다섯

	문법	하는 일
키워드	finally:	무조건 마지막에 실행 — 정리 전용
키워드	else:	성공했을 때만 실행
키워드	continue	이번 회차만 건너뛰고 다음으로
문법	except ValueError as e:	e에 예외 정보 — 무엇이 왜 실패했는지
키워드	raise ValueError("메시지")	내가 직접 예외를 일으킨다

좋은 예외처리 4원칙

원칙	나쁜 예	좋은 예
① 구체적으로	except:	except ValueError:
② 작게	전체를 try로 감쌌	위험한 줄만
③ 기록	개수만 세기	bad.append((id, str(e)))
④ 드러내기	전부 잡아 숨김	예상 못한 예외는 터지게 둔다

반복문 예외처리 — 외워둘 형태

이 한 덩어리에 오늘 배운 것이 다 들어 있다

```
bad = []
```

```
for i, row in enumerate(rows, start=1):
    try:
        value = float(row["측정값"])          # ① 위험한 줄만
        if value < low or value > high:
            raise ValueError(f"{value} 범위 초과") # ② 내 규칙
    except (ValueError, TypeError) as e:      # ③ 구체적 + as e
        bad.append((i, str(e)))               # ④ 기록
        continue                             # ⑤ 그 줄만 건너뛰

    valid.append(row)
```

```
print(f"정상 {len(valid)}건 · 불량 {len(bad)}건")
```

파이프라인 6단계

단계	함수	핵심	반환
1 읽기	<code>read_csv()</code>	next로 헤더 분리 · <code>FileNotFoundError</code> 대비	<code>header</code> , <code>rows</code>
2 분류	<code>group_by_part()</code>	없는 키면 빈 리스트부터	{부품: [행]}
3 통계	<code>calc_stat()</code>	숫자 아닌 값 건너뛰 · 0건이면 <code>None</code>	(개수, 평균, 최소, 최대)
4 방어	<code>filter_valid()</code>	<code>try·raise·as e·continue</code> 총동원	<code>valid</code> , <code>bad</code>
5 저장	<code>save_report()</code>	줄을 리스트에 모아 한 번에 쓰기	<code>report.txt</code>
6 검증	<code>verify_by_part()</code>	부품별 합 == 전체 정상 개수	<code>True</code>

리포트 3원칙

원칙	실천
① 핵심 먼저	전체 · 정상 · 불량 요약을 맨 위에
② 단위 붙이기	숫자마다 "도" · "개" 같은 단위와 설명을 함께
③ 구분선 활용	" - " * 40 으로 요약과 세부를 가른다

오류·함정 사전

한 번씩은 반드시 겪는 것들

B-1 · 오늘 실제로 겪은 것

X 이렇게 하면	무슨 일이	✓ 바로잡기
<code>except NameError: # float() 뒤</code>	도달 불가 — 절대 실행되지 않음	실제로 날 수 있는 예외만 적는다
<code>finally: print(결과)</code>	실패해도 결과처럼 보이는 값이 출력	성공 결과는 else, 정리는 finally
<code>except: # 종류 없음</code>	오타·진짜 버그까지 삼킴	<code>except ValueError as e:</code>
<code>total = (a+b+c)/3</code>	이름은 합계인데 값은 평균	<code>student_avg = ...</code>
<code>row.get("\uffeff이름")</code>	BOM을 코드에 하드코딩	<code>encoding="utf-8-sig"</code>
1단계가 return 없음	2단계가 파일을 다시 읽음	결과를 return해 이어받게
모든 열에 통계	검사ID 평균 같은 무의미한 값	분석 대상 열만 지정

B-2 · 예외처리 함정

X 이렇게 하면	무슨 일이	✓ 바로잡기
try가 반복문 바깥	첫 불량에서 전체 중단	반복문 안쪽 에 둔다
except에 continue 없음	실패해도 아래 코드가 실행 — 이전 값이 출력	<code>continue</code> 로 다음 회차
넓은 except가 위	아래 구체적 except는 도달 불가	구체적 → 넓은 순서
개수만 세기	무엇이 왜 빠졌는지 알 수 없음	<code>as e</code> 로 사유까지 기록
<code>raise</code>	잡지 않으면 프로그램이 멈춤	호출하는 쪽에서 try-except
with 블록 밖에서 f 사용	ValueError: I/O operation on closed file	필요한 값은 블록 안에서 변수에

B-3 · 파이프라인 함정

증상	원인	해결
0으로 나누기 오류	유효 데이터가 0건인데 평균 계산	<code>if count == 0: return None</code>

증상	원인	해결
최솟값이 항상 0	min을 0으로 초기화	None 으로 시작 · is None or <
빈 키가 생김	빈 행의 부품명이 "" 라서	분류 전에 빈 값 건너뛰기
개수 합이 안 맞음	중복 처리 또는 누락	6단계 검증으로 조기 발견
한글이 깨짐	인코딩 불일치 · BOM	utf-8-sig · cp949 시도

개념

부록 B의 절반이 여전히 **오류 없이 결과만 틀리는 것들**입니다 — 도달 불가 except, finally의 결과 출력, total이라는 이름, 무의미한 통계. try-except를 배웠다고 이런 게 잡히지는 않습니다. 오히려 **예외처리를 남용하면 이런 오류가 더 잘 숨습니다**. 그래서 교안이 4원칙 마지막에 "드러내기"를 넣은 것이고, 09_03이 6단계에 "검증"을 넣은 것입니다.

실습 하이라이트 & 다음 시간 예고

파이썬 기초의 마지막 회차

C-1 · 오늘 자료와 작성 파일

파일	내용	상태
Lecture/09_02_견고한_예외처리	else · finally · 리소스 정리 · 반복문 · bare except · as e · raise (27p)	전 범위
Lecture/09_03_데이터_처리_파이프라인	파이프라인 설계 + 6단계 종합 실습 (17p)	전 범위
09_finally_..._5.py	try-except-finally	도달 불가 except 1건
09_continue_..._6.py	반복문 + continue	bare except
10_student_scores_..._7_8.py	CSV 집계 시연 + 확장	이름 불일치 · 예외처리 없음
Practice/09_02_example.py	학생 점수 + 과목별 최고점	동작 정확
Practice/09_03_example.py	파이프라인 1~6단계	완주 · 검증 True

C-2 · 오늘 배운 것 한눈에

분류	오늘 나온 것	개수
키워드	finally · else · continue · raise	4개
문법	except ... as e	1개
원칙	bare except 회피 · 좋은 예외처리 4원칙 · 리포트 3원칙	3개
패턴	반복문 + try + continue · try-finally 정리 · 검증 함수 분리	3개
파이프라인	읽기 · 분류 · 통계 · 방어 · 저장 · 검증	6단계

C-3 · 파이썬 기초 완주

오늘로 파이썬 기초 문법이 완전히 끝났습니다. 그리고 그것들을 조립해 실제로 돌아가는 분석 프로그램까지 만들었습니다.

구간	회차	내용	오늘 파이프라인에서
값·연산	1~4	변수 · 자료형 · 연산자 · 입력	점수 · 측정값 계산
문자열	4~5	슬라이싱 · 메서드 · f-string	리포트 문장 만들기
제어 흐름	7~8	if · for · while · continue	행 순회 · 불량 건너뛰기
자료구조	6~10	리스트 · 튜플 · 셋 · 딕셔너리	2단계 분류 · 3단계 반환
함수	11	def · 매개변수 · return · 설계	6개 함수로 분리 · 재사용
모듈·파일	12~13	import · os · open · csv	1단계 읽기 · 5단계 저장
예외처리	13~14	try · except · finally · raise	4단계 방어

개념

오른쪽 열을 세로로 읽어보세요. **14회차 동안 배운 모든 구간이 오늘 파이프라인 어딘가에서 한 번씩 쓰였습니다.** 교안이 "배운 문법이 단계마다 한 번씩 등장한다"고 한 것이 과장이 아니었던 셈입니다. 그리고 이 프로그램은 그 **자체로 포트폴리오**가 됩니다 — "CSV를 읽어 불량을 걸러내고 통계를 내어 리포트로 저장하는 파이프라인을 설계했다"는 한 문장이 자기소개서에 그대로 들어갈 수 있습니다.

C-4 · 다음 시간 예고 — NumPy

다음 회차부터 **라이브러리 구간**으로 넘어갑니다. 교안 10_01 배열 생성과 구조가 그 시작입니다.

다음 주제	한 줄 미리보기
왜 NumPy인가	설비 센서 데이터는 하루에 수백만 개. 리스트에 숫자를 곱하면 계산이 아니라 반복 이 일어난다 — 배열은 한 줄로 해결
np.array · arange · linspace	리스트를 배열로 · 연속 정수 · 균등 분할 — 만드는 방법 세 가지
ndim · shape · size · dtype	괄호 없는 속성들 . 데이터를 만나면 가장 먼저 확인하는 네 가지
reshape · flatten	값은 그대로, 모양만 바꾸기 — 표를 한 줄로 퍼거나 한 줄을 표로

연결

오늘 만든 파이프라인이 **NumPy·Pandas로 넘어가는 다리**입니다. 3단계에서 손으로 짠 평균·최대·최소가 `arr.mean()` 한 줄이 되고, 4단계 조건 필터가 `arr[arr > 90]` 한 줄이 됩니다. **무엇을 대신해 주는지 알고 쓰는 것과 외워 쓰는 것의 차이가** 여기서 갈립니다.

오늘의 성취와 다음 한 걸음

부품에서 기계로

오늘 배운 것을 한 문장으로 줄이면 — **예외를 견고하게 다루고, 지금까지 배운 부품을 조립해 하나의 분석 프로그램을 만들었다.** 14회차 동안 따로따로 배운 문법들이 오늘 처음으로 **한 덩어리로 맞물려 돌아갔습니다.**

그리고 제출하신 09_03_example.py는 그 조립이 실제로 작동한다는 증거입니다. 6단계를 함수로 나누고, 뒤 단계가 앞 단계를 재사용하고, 마지막에 개수 검증까지 통과했습니다. **수업을 놓쳤다고 하셨지만 코드는 제대로 서 있습니다.**

오늘 헛갈렸던 지점 다시 짚기

헛갈린 것	정리
else 와 finally	except와 else는 정확히 반대(둘 중 하나만), finally만 양쪽 다 실행
finally에 무엇을 넣나	정리만. 파일 닫기·연결 종료. 성공 결과는 else로
try를 어디에 두나	반복문 안쪽 . 바깥에 두면 첫 불량에서 전체가 멈춘다
왜 bare except가 나쁜가	오타·진짜 버그까지 삼켜 조용히 잘못된 결과를 낸다
as e 는 왜 쓰나	"문제 발생"이 아니라 무엇이 왜 실패했는지 기록하려고
raise 는 언제 쓰나	파이썬은 정상이라 보지만 현실 규칙상 잘못된 값일 때 (-999도 같은)
파이프라인이 뭐가	앞 단계 결과가 뒤 단계 재료가 되는 구조. 수도관처럼 이어진다
왜 단계로 나누나	오류 난 단계를 찾기 쉽고, 한 단계만 고쳐도 되고, 함수를 재사용 할 수 있어서

개념

복습 루틴 추천: ① 09_finally 코드에서 text를 정상값과 비정상값으로 각각 바꿔 finally가 양쪽 다 실행되는 것을 **눈으로 확인** → ② 09_continue의 try를 반복문 **바깥으로 옮겨** 결과가 어떻게 달라지는지 보기 → ③ except:를 except ValueError as e:로 바꾸고 print(e) 추가해 사유가 어떻게 나오는지 확인 → ④ 09_03_example.py의 print_col에 return header, rows를 추가하고 2단계가 그걸 받아 쓰도록 고쳐 보기 → ⑤ 리포트 [불량 목록]을 원인별로 세어 "결측 몇 건 · 범위 초과 몇 건"으로 요약해 보기. ④⑤는 이미 완성한 코드를 한 단계 더 다듬는 것이라 부담이 적습니다.